

How the Black Scholes Formula uses probability theory

◆ This title was summarized by AI from the post below.



Charlie Browne

10mo



The Black Scholes Formula

Differential equations in physics model movement in space and time. Newton's laws, the heat equation and the wave equation are examples. Quantitative finance borrowed heavily from these equations to model price & rate movements across a screen over time. Price movement is random so the spaces that they move in are probabilistic, following certain patterns depending on the type of financial variable. The patterns can be represented by mathematical formulas known as probability density functions (PDFs). Some probability distributions are easier to work with than others. Solutions to differential equations that provide estimates for a price or an option on a price at a particular point in time are much easier to achieve if price distributions are bell-shaped and



12,563 followers

[86 Posts](#) · [2 Articles](#)

[View Profile](#)

[+ Follow](#)

More from this author



FRTB Breakfast Briefing

Charlie Browne · 8y



GoldenSource at V-Fi Europe

Charlie Browne · 8y

LinkedIn



Top Content



People



Learning



Jobs

[Sign in](#)

[Join now](#)

K, i.e. the amount above the strike K that the price S finishes. During the life of the

option, the at-maturity value of S-K can be modelled by simulating paths of S. The simulation will result in “densities” of prices at maturity. The more paths than finish at a given price level, the higher its density at that level. In mathematical terms, this simulation is defined as an integral. The integral is the sum of all the possible payoffs S-K multiplied by probabilities that the payoffs occur. The probabilities are determined by the path densities at maturity.

Running Monte Carlo simulations to estimate price densities at maturity is computationally expensive. If a single formula can be used to calculate the option value, it is preferred over simulation or other numerical approaches. Single formulas, however, are typically only possible if the price PDF is normal, i.e. bell shaped and symmetric.

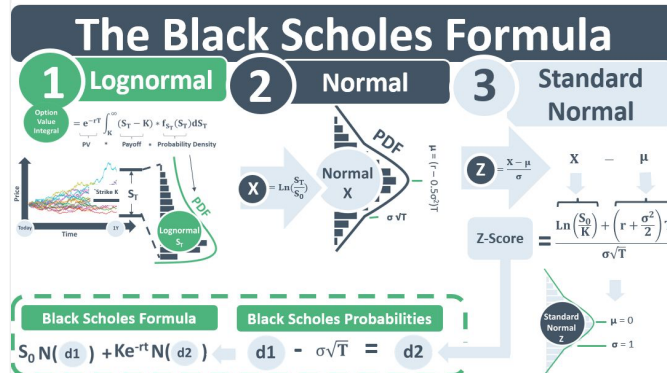
The standard normal distribution (SND) is the easy-to-work-with PDF that the BSM formula uses. It assumes that that densities of S, when they arrive at option-maturity, are bell shaped, with a mean of 0 and a standard deviation of 1. The Z-scores of the SND are used by the BSM formula. They are directly derived from the formula’s famous d_1 and d_2 parameters. $N(d_1)$ and $N(d_2)$ represent probabilities of the option being exercised.

The simulated stock prices shown under 1) below result in a lognormal distribution of prices. It is lognormal because stock prices cannot go below zero and do not have an upper bound. Lognormality implies asymmetry. The data is not bell shaped. While formulas exist for deriving probabilities from lognormal data, they are

Explore content categories

[Career](#)[Productivity](#)[Finance](#)[Soft Skills & Emotional Intelligence](#)[Project Management](#)[Education](#)[Show more ▼](#)

difficult to work with. The BSM formula, therefore, needs to perform two transformations to arrive at the easy-to-work with SND PDF shown under 3. The first is a transformation from a lognormal probability space to a normal one. This is the transformation from S to X btw 1 and 2 in the diagram. The second is the transformation from X to the SND space, Z.



👍💡🗨️ 201 · 6 Comments

👍 Like

🗨️ Comment

➦ Share



SANKARA NARAYANAN.V

9mo



Thank you for the insightful post.
Just a small observation — in the Black-Scholes formula for a European call option, the second term (related to the strike price) is typically subtracted, not added, from the first term (related to the spot price).
That is, the formula reads:

$$C = S_0 \cdot N(d_1) - K \cdot e^{-rT} \cdot N(d_2)$$

a typographical oversight — everything else is clearly and thoughtfully presented.
Appreciate the effort in demystifying such a complex topic.

Like · Reply | 3 Reactions

To view or add a comment, [sign in](#)

More Relevant Posts



igor kolesnikov

7mo



A Single Equation That Proves $P = NP$ —
Using Photonic Computation

What if the deepest question in computer science could be resolved not through brute-force logic, but through the elegance of wave mechanics?

I propose a radically simple equation:

$w = f(w)$

$w = f$

Where:

w is a wave representing all possible solutions to an NP-complete problem (e.g., 3-SAT),

$f(w)$ is a physical operator that verifies those solutions — implemented optically on a photonic computer.

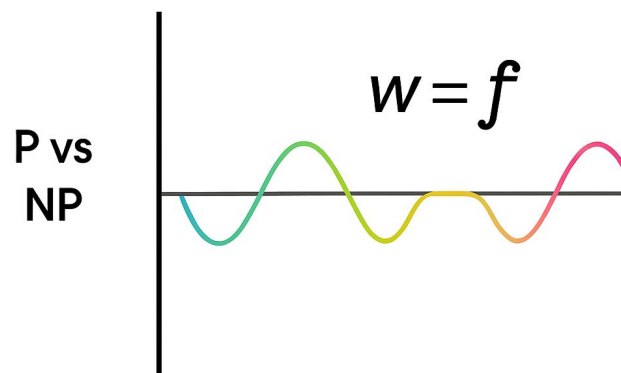
If the wave remains unchanged after verification — that is, if $w = f$ — then the solution is self-confirming. The act of solving and the act of verifying become indistinguishable.

This is not just symbolic. On a photonic computer, all possible solutions can exist simultaneously as a superposition. The verification operator (e.g., an optical filter or interference pattern) eliminates invalid paths. If the output wave equals the input, the system has found and verified the solution in one physical step.

In this framework, $P = NP$ is not a theoretical abstraction — it's a measurable symmetry.

This equation is more than math. It's a bridge between quantum logic, computational complexity, and physical reality. Elegant. Deterministic. Beautiful.

©



17 · 9 Comments

Like

Comment

Share

To view or add a comment, [sign in](#)

MQL5 Algo Trading

454 followers

6mo

In computational tasks, arrays serve as the backbone for ordered data collections, enabling efficient access to elements using indices. They are crucial in solving linear algebra, mathematical modeling, and machine learning problems through operations on matrices and vectors. Such tasks demand mathematical proficiency and the ability to manage complex loops. Advanced data types like 'matrix' and 'vector' facilitate code that resembles mathematical notation, reducing reliance on nested loops.

In MQL5, vectors are one-dimensional, comprising double-type arrays, and support operations like addition, multiplication, and norm calculation. Conversely, matrices extend this structure into two dimensions. Operations such as element-wise arithmetic, matrix multiplication, and various forms of decomposition are supported, including LU, QR, and SVD. Understanding t...

[#MQL5](#) [#MT5](#) [#Algorithm](#) [#Vector](#)

<https://lnkd.in/d47hSjNc>

Like

Comment

Share

To view or add a comment, [sign in](#)

GyaanSetu PHP

134 followers

7mo

2221. Find Triangular Sum of an Array

2221. Find Triangular Sum of an Array

Difficulty: Medium

Topics: Array, Math, Simulation,
Combinatorics, Biweekly Contest 75

You are given a 0-indexed integer array
nums, where nums[i] is a digit between 0
and 9 (inclusive).

The triangular sum of nums is the value of
the only element present in nums after the
following process terminates:

Let nums comprise of n elements. If $n == 1$,
end the process. Otherwise, create a new
0-indexed integer array newNums of
length $n - 1$.

For each index i, where $0 \leq i < n - 1$,
assign the value of newNums[i] as $(\text{nums}[i] + \text{nums}[i+1]) \% 10$, where % denotes
modulo operator.

Replace the array nums with newNums.

Repeat the entire process starting from step 1.

Return the triangular sum of nums.

Example 1:

Input: nums = [1,2,3,4,5]

Output: 8

Explanation: The above diagram depicts the process from which we obtain the triangular sum of the array.

Example 2:

Input: nums = [5]

Output: 5

Explanation: Since there is only one element in nums, the triangular sum is th

<https://lnkd.in/giBQSHae>

Like

Comment

Share

To view or add a comment, [sign in](#)

Stéphane Dalbera

7mo

I often talk about my love for the FFT, and today I would like to discuss one of its uses that can be surprising and is often little known: the multiplication of large numbers in linearithmic time ($O(n \log n)$).

Multiplying two large numbers, such as those with hundreds or thousands of digits, is computationally demanding.

The traditional algorithm for multiplication, similar to what we learn in school, operates with a complexity of $O(n^2)$, where n is the number of digits.

For extremely large numbers, this approach becomes inefficient.

The FFT-based approach stems from a clever mathematical observation: circular convolution.

Multiplying two large numbers can be viewed as multiplying two polynomials, where each digit of the number corresponds to a coefficient of the polynomial.

The product of these two polynomials results in a convolution of their coefficients.

And as convolution in the time domain (or coefficient domain) corresponds to simple multiplication in the frequency domain... FFT is the solution.

The FFT allows transforming a polynomial from the coefficient domain to the frequency domain :

- Transform the polynomials representing the two large numbers into their frequency domain representation using the FFT (complexity $O(n \log n)$).
- Multiply the frequency representations of the two polynomials (linear complexity $O(n)$).
- Apply the inverse FFT to return to the original representation (complexity $O(n \log n)$).

That's it : the quadratic complexity $O(n^2)$ is transformed in linearithmic complexity $O(n \log n)$.

65 · 11 Comments

Like

Comment

Share

To view or add a comment, [sign in](#)

Ashish Madaan

6mo

How do you describe a perfect circle to a computer that only understands straight lines and numbers?

You don't. You approximate it.

That clever approximation is the foundation of all modern CFD.

Last week, we met the Navier-Stokes equations - the fundamental "laws" of fluid flow.

The problem? They are continuous equations, meaning they apply at an infinite number of points in space.

Computers can't handle infinity. They excel at finite arithmetic.

So, how do we solve them? We translate the problem from the language of continuous calculus into the language of finite algebra.

This translation is Discretization, and it happens in two steps:

 Discretize Space: We break our physical geometry (like a pipe or a wing) into a finite number of small, simple pieces called "cells." This grid of cells is the Mesh

 Discretize the Equations: We convert the calculus equations into simple algebraic equations that can be solved for each individual cell.

The computer then solves this massive, interconnected system of equations, ensuring that the mass, momentum, and energy are all conserved as they move from one cell to the next.

 Think of a digital photograph. Reality has infinite color and detail. Your camera captures it by dividing the scene into pixels each one holding a single color value. With enough pixels, the image looks perfectly smooth. A CFD mesh works the same way - more cells mean higher fidelity and more realistic results.

 The "order" of your discretization scheme (the math used for the translation) matters. First-order: Fast and stable, but coarse like low-resolution pixels.

Second-order: Sharper and more accurate standard in engineering.

[#CFD](#) [#Discretization](#)

[#NumericalMethods](#) [#Simulation](#)

[#FiniteVolume](#) [#Engineering](#) [#Physics](#)

[#Computation](#) [#DigitalEngineering](#)

45

Like

Comment

Share

To view or add a comment, [sign in](#)

Kalman Knizhnik

6mo · Edited

Modeling complex, stochastic physical systems was the primary task of the first 15 years of my career.

One of the things I learned doing this is that such models should primarily be interpreted qualitatively, and quantitative results should be approached very

skeptically.

As I describe here, how you initialize your model, as well as the assumptions you implicitly put into it matter a great deal, and will strongly affect your results.

https://lnkd.in/ey_RRaBu

Models and Their Flaws

reflectionsofbelief.substack.com

27 · 3 Comments

Like

Comment

Share

To view or add a comment, [sign in](#)

Ramesh Doddi

7mo

◆ Binary Search on Answers — Part 1
Sharing two foundational numeric search problems that use binary patterns to replace slow iteration.

📌 This covers:

◆ FindingSqrt — compute $\text{floor}(\text{sqrt}(n))$.
We compare brute $O(n)$ scanning vs a binary-search approach ($O(\log n)$) that narrows candidate roots. Great for limiting search in integer math and fast root approximations.

◆ Nthroot — find integer b such that $b^n == m$. Brute force tries bases; the binary approach uses fast exponentiation + binary search on base. Useful when checking exact integer roots or validating power

relationships.

 Real-world parallels:

👉 Numeric scaling & approximation — fast root checks are useful in graphics (scale factors), physics simulations, and numeric libraries where repeated square-root searches are common.

👉 Data integrity & validation — exact power checks appear when validating encoded/hashed values, special-purpose compression checks, or discrete math validations.

[#DSA](#) [#BinarySearch](#) [#ProblemSolving](#)
[#Algorithms](#)

1

Like

Comment

Share

To view or add a comment, [sign in](#)